Scaling the Kitaev/VQE Simulation to Large L

From the analysis to the built pipeline: free fermions, statevectors, MPS trajectories, and device-calibrated noise

Seminar notes

August 10, 2026

Abstract

Can the Block-4 noise study be pushed from $L = 4$ toward $L = 16$ (and beyond)? This note works through the question and then documents the pipeline that was actually built to answer it (`src/free_fermion.py`, `src/block4_scaling.py`, `scripts/lsweep.slurm`). The original Block-4 evaluation hides *two* exponential objects. The **exact-diagonalization (ED) reference** is fully removable: the Kitaev Hamiltonian is free-fermion, so its ground state and the edge string follow from a $2L \times 2L$ Bogoliubov–de Gennes (BdG) matrix in $O(L^3)$, exact to $L \sim$ hundreds. The **noisy VQE-prepared state** is the hard one; we remove its 4^L density matrix with quantum-trajectory sampling on a matrix-product-state (MPS) backend (2^L, not 4^L), and we compute the *ideal* VQE expectations from a 2^L statevector with sparse Pauli operators (never the $2^L \times 2^L$ dense Hamiltonian). Along the way several implementation choices proved essential and are documented in full: the working point $\mu = 0.5t$ rather than the degenerate sweet spot $\mu = 0$; an **L-BFGS-B** optimizer with an ED-free convergence oracle in place of COBYLA; even-parity–sector state selection without ED; a device noise model that transplants *real* IBM calibration onto the native linear chain; and a process-pool parallelisation of the (L, reps) grid. The scientific payoff is a clean, honest result at $L \leq 16$: for the edge-string diagnostic the two failure modes *decouple*. Expressibility is cheap and L-independent — a *constant* depth $r^* = 4$ reproduces $\langle X_0 Z_1 \cdots Z_{L-2} X_{L-1} \rangle$ all the way to $L = 16$ — so the large-L wall is **noise alone**: at fixed depth the CNOT count grows linearly and decoherence erases the diagnostic.

Contents

Part I
The analysis: where the exponentials hide

1 Two exponential objects

The original Block-4 evaluation (`src/block4.py`, plots 8–9) contains two objects that scale exponentially in the number of qubits L:

1. the **ED reference** — `np.linalg.eigh` on the $2^L \times 2^L$ Hamiltonian, used for the ideal $\langle X_0 Z_1 \cdots Z_{L-2} X_{L-1} \rangle$ curve, the parity gap, and the fidelity post-selection;

2. the **noisy state** — an exact density matrix ρ of dimension $2^L \times 2^L$ (i.e. 4^L complex entries), produced by `save_density_matrix` on the Aer `density_matrix` method.

The observables themselves are *not* the problem: `qubit_hamiltonian`, `edge_string`, and `parity` are already `SparsePauliOp` objects with $O(L)$ Pauli terms. The blow-up comes entirely from calling `.to_matrix()` on them and from `eigh`. The two objects are addressed separately below, and both are removed in the built pipeline (Part II).

2 The dense wall

Object 2 sets the practical ceiling. A dense $2^L \times 2^L$ complex128 matrix costs $16 \cdot 4^L$ bytes, and the pipeline holds several copies (ρ, H, $\hat{P}$, ED eigenvectors, Aer's internal state):

L	density matrix	+ copies	ED `eigh` cost
12	0.27 GB	∼1 GB	seconds
14	4.3 GB	∼20 GB	minutes
15	17 GB	∼70 GB	∼10 min
16	69 GB	∼300 GB	hours

Worse, *every* VQE cost evaluation in the dense path is a full 4^L density-matrix simulation, run maxiter × n_starts times per L. Beyond $L \approx 14$ a single L-point becomes hours to days. This is the "dense wall" at $L \approx 15$, and it is the wall the new pipeline is built to beat.

3 Escaping object 1: free fermions (the idea)

The Kitaev Hamiltonian is quadratic in fermions. After a Jordan–Wigner map the qubit form is

$$H = -\tfrac{\mu}{2} \sum_j Z_j + \tfrac{t-\Delta}{2} \sum_j X_j X_{j+1} + \tfrac{t+\Delta}{2} \sum_j Y_j Y_{j+1},$$

a free-fermion (BdG) model. Its ground state, spectrum, covariance matrix, and the parity gap $\Delta_P(L) \sim e^{-L/\xi}$ all follow from a $2L \times 2L$ BdG matrix in $O(L^3)$ — no exponential object. The edge operator $X_0 Z_1 \cdots Z_{L-2} X_{L-1}$ is a product of two Majorana operators, so its ground-state expectation is a single entry of the covariance matrix (Wick's theorem). **The ED reference is removable outright**; the construction is `src/free_fermion.py` (§7).

4 Escaping object 2: trajectories and MPS (the idea)

4.1 Quantum-trajectory statevector sampling

Noise (a mixed state) does not require the full density matrix. In the quantum-trajectory picture each shot is a *pure* statevector (2^L, not 4^L) in which the Kraus operators are sampled stochastically; averaging an observable over N_{traj} trajectories reconstructs $\mathrm{Tr}(O\rho)$ up to a statistical error $\propto 1/\sqrt{N_{\text{traj}}}$. Memory drops from 4^L to 2^L (∼1 MB at $L = 16$), at the honest cost of *error bars*.

4.2 Matrix product states

An MPS factorises the 2^L-amplitude tensor into a chain of small tensors linked by bond indices of dimension χ, with memory $O(L\chi^2)$. Across any cut χ equals the number of nonzero Schmidt coefficients — a direct measure of entanglement. A gapped 1D state (area law) has χ bounded independent of L and MPS is *linear* in L; a volume-law state needs $\chi \sim 2^{L/2}$. A two-qubit gate can double χ across its bond; truncating to $\chi_{\max}$ Schmidt values discards a quantifiable *weight* (the sum of squared discarded coefficients), so truncation error is a logged, controllable knob rather than a hidden bias. If nothing is discarded the MPS is exact.

5 The χ-probe: what the real ansatz does

We measured the exact (uncapped) max bond dimension of the real `EfficientSU2(ry, linear, reps)` state.

χ is flat in L at fixed depth. Worst-of-random-θ max χ (reps $= 3$): $\chi = 8$ for all $L \in \{6, 10, 14, 18, 22\}$, far below the volume-law ceiling $2^{\lfloor L/2 \rfloor} = 8, 32, 128, 512, 2048$. A reps $= r$ linear ansatz has a light cone of width $\sim r$, so entanglement is bounded independent of L; noise does not inflate it. Cap $\chi_{\max} = 16$ and the simulation is exact to $L = 100$.

χ grows exponentially in depth. At $L = 14$: reps $= 1 \ldots 8 \Rightarrow \chi = 2, 4, 8, 16, 32, 64, 128$, i.e. $\chi \approx 2^{\text{reps}}$ until it saturates the $2^{L/2}$ ceiling.

The fixed-depth ansatz loses the *full* state. VQE *full-state* fidelity to the true ground state at reps $= 3$ (measured at $\mu = 0$): $0.99, 0.50, 0.25, 0.10, 0.02$ over $L = 4, 6, 8, 10, 12$. A fixed-depth hardware-efficient ansatz cannot represent the whole topological ground state as L grows.

Reconciliation (important). This full-state collapse is real, but it does *not* by itself decide the fate of the *edge-string diagnostic*, which is a *localised* two-Majorana quantity. Part III shows that $\langle X_0 Z_1 \cdots Z_{L-2} X_{L-1} \rangle$ is in fact recovered at a *constant* depth $r^* = 4$ up to $L = 16$. So: the full wavefunction needs growing depth (this section), but the specific observable the experiment measures does not. The two statements are consistent because a shallow circuit can build the two localised edge operators without reproducing the entire bulk.

6 The MPS wall is fundamental for the full state

Combining the probes: MPS is cheap where the ansatz has low depth (constant χ), which is exactly where the VQE *full-state* fidelity $\to 0$; and repairing fidelity by adding depth drives $\chi \sim 2^{\text{reps}}$ back up. If the required reps grows linearly with L, this is $\chi \sim 2^{O(L)}$. MPS buys a smaller exponential base (2^{reps} vs 4^L), a few extra qubits, not unbounded L — the honest simulation-cost face of "NISQ does not inherit topological protection." The one unconditionally polynomial regime is a matchgate/Gaussian ansatz with fermion-Gaussian noise ($O(L^3)$ covariance algebra), but that answers a cleaner *different* question and is not the generic hardware-efficient story.

Part II
The built pipeline: implementation

Everything below is implemented and validated. The reference is `src/free_fermion.py`; the study is `src/block4_scaling.py`; the HPC wrapper is `scripts/lsweep.slurm`.

7 The free-fermion BdG solver

7.1 Majorana Hamiltonian

With Majorana operators $\gamma_{2j} = (\prod_{k<j} Z_k)X_j$ and $\gamma_{2j+1} = (\prod_{k<j} Z_k)Y_j$ the qubit Hamiltonian becomes $H = \frac{i}{2}\sum_{a<b} A_{ab}\,\gamma_a\gamma_b$ with A real antisymmetric, $2L \times 2L$. Its nonzero couplings (from the Jordan–Wigner map) are

$$A_{2j,\,2j+1} = \mu, \qquad A_{2j+1,\,2j+2} = \Delta - t, \qquad A_{2j,\,2j+3} = t + \Delta,$$

plus antisymmetric partners.

7.2 Canonical form: why `eigh(iA)`, not real Schur

The energy needs the canonical frequencies $\varepsilon_k \geq 0$ with $A = O\bigoplus_k \varepsilon_k \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right) O^\top$ and $E_0 = -\frac{1}{2}\sum_k \varepsilon_k$. The first implementation used `scipy.linalg.schur(A, output='real')` and read the 2×2 blocks. This **silently breaks at large** L: the topological near-zero edge modes underflow machine precision and the real-Schur block extraction mis-assigns them, giving a wrong energy (at $L = 80$, $E = -19.3$ instead of ≈ -83). The fix diagonalises the Hermitian matrix iA instead: its eigenvalues are $\pm\varepsilon_k$, and for a positive eigenvalue ε with unit eigenvector $v = a + ib$ one has $Aa = \varepsilon b$, $Ab = -\varepsilon a$ and $|a| = |b| = 1/\sqrt{2}$, $a\cdot b = 0$ (because $v^\top v = 0$, v being orthogonal to its $-\varepsilon$ partner v^*). The orthonormal columns $(\sqrt{2}\,\mathrm{Im}\,v,\ \sqrt{2}\,\mathrm{Re}\,v)$ give the canonical block. This route has no block-guessing and is stable: the energy is now correct and scales linearly to $L = 160$ and beyond.

7.3 Ground state, covariance, and the edge string

The ground state fills all negative modes; its Majorana covariance is $M_{ab} = i\langle\gamma_a\gamma_b\rangle = O\,(\bigoplus_k -\left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right))\,O^\top$ (the block sign is $\langle i\gamma\gamma\rangle = -1$ in the ground state; getting this sign wrong flips all observables, which the self-test caught). The key identity is

$$X_0Z_1\cdots Z_{L-2}X_{L-1} = -\,i\,\gamma_1\gamma_{2L-2} \quad\Longrightarrow\quad \langle X_0Z_1\cdots Z_{L-2}X_{L-1}\rangle = -\,M_{1,\,2L-2},$$

a *single* entry of the covariance matrix, and $\langle Z_s\rangle = -M_{2s,2s+1}$.

7.4 Self-test

`free_fermion.py` self-tests against the exact `eigh` path (`block3_core`) at *gapped* $\mu \in \{0.5, 1.0, 2.5\}t$, $L \in \{4, 6, 8\}$: energy agrees to $\sim 10^{-15}$, and the edge string, $\langle Z_0\rangle$, and the many-body parity splitting all match. The parity gap decays as $e^{-L/\xi}$ (1.8×10^{-6} at $L = 10$, 1.7×10^{-12} at $L = 20$, below machine precision by $L = 40$). The test deliberately avoids $\mu = 0$ — see §10.

8 The ideal path: statevector, not `to_matrix`

The ideal (noiseless) VQE expectations of H, $\hat{P}$, and $X_0Z_1\cdots Z_{L-2}X_{L-1}$ are computed as $\langle\psi(\theta)|\,O\,|\psi(\theta)\rangle$ with a 2^L `Statevector` and the `SparsePauliOp` directly (`Statevector.expectation_value`). This never forms the $2^L\times 2^L$ dense H, so it is exact and dramatically faster than `.to_matrix()`: at $L=14$, 6.6 ms versus 3357 ms ($\sim$500$\times$), agreeing to $\sim 10^{-15}$. A 2^L statevector is trivial to $L\sim 22$ (16 MB at $L=20$); it is the mild, unavoidable object (and MPS- compresses to $\chi\sim 2^{\text{reps}}$ beyond that). *This is why the dense-`eigh` scan was slow and the statevector scan is fast — it is the same physics without the 4^L matrix.*

9 The noisy path: MPS trajectories with honest error bars

The noisy edge string is `AerSimulator(method='matrix_product_state', noise_model=...)` run for N_{traj} shots; each shot is one stochastic-Kraus trajectory (a pure state, 2^L, $\chi\le 2^{\text{reps}}$). We read `save_expectation_value_variance`, which returns [mean, var] with `var` the single-trajectory variance of $X_0Z_1\cdots Z_{L-2}X_{L-1}$ (≈ 1 for a Pauli); the standard error on the mean is $\sqrt{\text{var}/N_{\text{traj}}}$.

Validation. Against the exact density matrix the estimator is unbiased and converges as $1/\sqrt{N}$: the difference falls $0.030\to 0.0008$ as $N=1\to 20000$. At reps $=3$ the MPS and statevector trajectories are bit-identical (no truncation). **Caveat found the hard way:** `seed_simulator` does *not* decorrelate re-runs on this Aer path, so batch-averaging independent seeds badly underestimates the error; the honest way to shrink the bar is to raise N_{traj}, and the variance instruction gives the correct bar in one run.

10 Choosing the working point: $\mu = 0.5t$, not $\mu = 0$

The Week-9 decks call $\mu=0$ the "sweet spot," but it is the one point where the edge-string reference is *ill-defined*. At $\mu=0$ with $t=\Delta$ the two end Majoranas γ_1,γ_{2L-2} are *exact* zero modes: the BdG ground state is parity-degenerate ($\Delta_P=0$), the two zero modes are decoupled, and the covariance entry $M_{1,2L-2}=0$. The free-fermion solver therefore returns $\langle X_0Z_1\cdots Z_{L-2}X_{L-1}\rangle=0$ at $\mu=0$ — correct for the symmetric vacuum, but useless as a reference. The VQE inherits the same pathology: with a vanishing parity gap the even and odd sectors are near-degenerate, so an energy minimiser lands on even/odd *superpositions* whose edge string cancels to ≈ 0. (An early sweep at $\mu=0$ produced exactly this: erratic $\langle X_0Z_1\cdots Z_{L-2}X_{L-1}\rangle\in\{0.0, 0.1, 0.008,\ldots\}$.)

The cure is to work at a *gapped* topological point, $\mu=0.5t$ ($|\mu|<2t$). There the free-fermion edge string is well-defined and stable ($\langle X_0Z_1\cdots Z_{L-2}X_{L-1}\rangle\to 0.9375$, saturating with L), the parity gap is small but nonzero ($\sim 10^{-3}$ at $L=4$ down to $\sim 10^{-10}$ at $L=16$), and both the reference and the VQE are clean. $\mu=0.5t$ is the default everywhere in `block4_scaling.py`.

11 The VQE optimiser

11.1 Cost and ED-free state selection

The cost is $\langle H\rangle+\lambda(1-\langle\hat{P}\rangle)^2$ on the statevector. The parity penalty must be *firm*: at $\mu=0.5t$ the parity gap is $\sim 10^{-4}$, so a weak penalty ($\lambda=0.1$) lets the optimiser drift into superpositions with $\langle X_0Z_1\cdots Z_{L-2}X_{L-1}\rangle\approx 0$; $\lambda=1.0$ pins the even sector. The old `block3_core` pipeline selected the best of several starts by *ED fidelity* — unavailable at large L. We replace it with an ED-free rule:

among the restarts, keep the lowest-energy optimum that sits in the even sector ($\langle \hat{P} \rangle \geq 0.5$), which is the correct variational target (the lowest-energy even-parity state the depth-r ansatz can make).

11.2 COBYLA → L-BFGS-B

The original optimiser was COBYLA. It **silently fails above ∼30 parameters**: at $L = 10$, reps = 5 (60 parameters, true edge 0.939) COBYLA with 8 starts returned 0.034 (energy 0.95 above the ground state). Gradient information fixes it. The comparison (same cell):

optimiser	starts	edge	ΔE
COBYLA	8	0.034 *(fail)*	+0.954
COBYLA	20	0.935	+0.068
L-BFGS-B	8	0.939	+0.033
L-BFGS-B	20	0.939	+0.033

L-BFGS-B is reliable with only 8 starts (better than COBYLA with 20), so it is the optimiser used throughout.

11.3 Free-fermion energy as an ED-free convergence oracle

Because the exact ground energy E_{ff} is available for free (§7), we use it to stop restarting: as soon as an even-sector optimum comes within `conv_tol` = 0.1 of E_{ff} the search halts. Easy cells converge on the first start; genuinely hard or *inexpressible* cells (too-small r) exhaust `n_starts` and honestly report the best the ansatz managed. This both speeds the grid and keeps the "failure" cells honest.

11.4 A warm start that does *not* work

We tried seeding depth r from the padded depth-$(r-1)$ solution. It has no effect: `EfficientSU2`(reps = r) carries an *extra fixed CX entangler* relative to reps = $r-1$, so padding a shorter solution with a zero rotation layer does *not* reproduce the shallower state — the extra entangler scrambles it. The idea was dropped; random restarts with L-BFGS-B are what is used.

11.5 Cross-check against ED

To be sure the "collapse" cells are physics and not a broken optimiser, we ran `block3_core.best_state` (dense, ED-fidelity selection, 20 starts) at $L = 6$, reps = 3: it also returns $\langle X_0 Z_1 \cdots Z_{L-2} X_{L-1} \rangle = 0.000$ (fidelity 0.53). The matrix-free optimiser agrees with the ED ground truth.

12 Device-calibrated noise: real hardware parameters

The toy model is a uniform per-`cx` depolarising channel ($p_{cx} = 0.05$). To use *real* hardware parameters one cannot simply call `NoiseModel.from_backend` on the linear-chain circuit: the real ≥ 16-qubit devices are *heavy-hex* (and often ECR/CZ-native), so our `cx`$(i, i{+}1)$ gates are *not* calibrated edges and would receive *no* two-qubit noise. We verified this on `FakeCairoV2`: its `from_backend` model has `cx` errors only on the 12 real device edges, none of which are $(0, 1), (1, 2), \ldots$.

`device_noise_model` therefore *transplants* the genuine recorded channels onto the native chain: the real per-qubit single-qubit (sx) error channels for qubits $0 \ldots L{-}1$ as-is, and a representative (median-error) real two-qubit channel on every `cx`. Real `FakeCairoV2` numbers: $T_1 \approx 97\,\mu$s, $T_2 \approx 81\,\mu$s, $1q$ error 2.7×10^{-4}, $2q$ error 7.5×10^{-3}, $2q$ duration 236 ns — so the real $2q$ error is ∼6× *smaller* than the toy 0.05.

Two honest options. (A) *Full device transpilation* routes the circuit onto the real topology with SWAPs and uses `from_backend` directly — "what you'd get on Cairo today," but dominated by routing overhead. (B) *Real parameters on the native chain* (the transplant above) — isolates the *intrinsic* device noise from the compiler. We use (B); it is what "real hardware parameters" most directly means and keeps the clean scaling narrative. `--backend FakeCairoV2` (27q) or `FakeWashingtonV2` (127q, for $L > 27$) selects it.

13 Parallelising the grid

The (L, reps) optimisations are independent, so `--workers N` runs them across a spawned `ProcessPoolExecutor`. Each worker is pinned to a *single* BLAS thread (thread-limit env set before the pool, inherited by the spawned children), because one L-BFGS-B optimisation is serial and its 2^L statevectors are too small for BLAS threading to scale — oversubscription actively slows it. So N workers $\times 1$ thread fill the node cleanly; the fast noisy trajectory phase then runs serially in the main process, where Aer reclaims all threads. Empirically: the $L = 4 \ldots 10$ Cairo grid on 8 workers took 7 minutes ($\sim 6\times$ efficiency). Live `[k/N]` progress is printed as each cell lands.

14 Reproducibility

- `python src/block4_scaling.py --selftest` — validates both primitives (statevector vs dense $\sim 10^{-15}$; trajectory vs density matrix $\sim 1\sigma$).
- `--Lmax N [--backend FakeCairoV2] [--workers K]` — runs the scan, caches the data (`plots/block4_scaling_data[_tag].npz`), and renders the figure; `--out-tag` keeps toy and device runs from overwriting each other.
- `--replot [--out-tag ...]` — redraws from the cache without recomputing (restyle, or replot an HPC run locally).
- `[BACKEND=FakeCairoV2] sbatch scripts/lsweep.slurm` — the Leipzig SC cluster wrapper (`paul` CPU partition; `module load` + user-site qiskit), `WORKERS` defaulting to `cpus-per-task`.

Part III
Results

15 The two failure modes decouple

The $L = 16$ cluster run (toy $p_{cx} = 0.05$, reps family $2 \ldots 7$) gives the noiseless edge table

$r \backslash L$	4	6	8	10	12	14	16
2	0.93	0.01	0.00	0.00	0.00	0.00	0.00
3	0.93	0.00	0.00	0.00	0.00	0.00	0.00
4	0.93	0.97	0.94	0.97	0.94	0.97	0.97
5	0.93	0.94	0.97	0.97	0.97	*0.00*	0.94
6	0.93	0.94	0.94	0.94	0.94	0.97	*0.01*
7	0.93	0.94	0.93	0.93	0.97	0.97	*0.00*

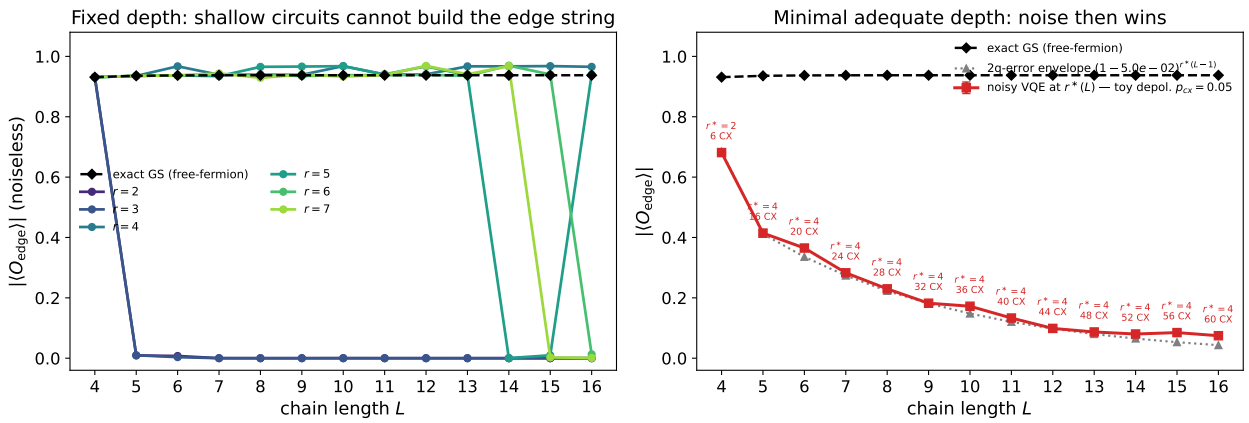

Figure 1: Two-panel scaling figure ($\mu = 0.5t$; toy $p_{cx} = 0.05$; grid run to $L = 16$ on the cluster). **Left:** noiseless edge string for a family of fixed depths r, with the exact free-fermion truth. Shallow $r = 2, 3$ fail beyond $L = 4$; a constant $r \geq 4$ tracks the truth to $L = 16$. (The dips at $r = 5, L = 14$ and $r = 6, 7, L = 16$ are optimiser stalls at large parameter counts, not physics — a smaller r succeeds there.) **Right:** at the minimal adequate depth $r^*(L)$, the noisy trajectory edge string decays along the $(1-p)^{n_{cnot}}$ envelope as the CNOT count grows.

so the minimal adequate depth is $r^*(L) = [2, 4, 4, 4, 4, 4, 4]$ for $L = 4 \ldots 16$. The italic entries are the optimiser stalls (a smaller r works at the same L).

Expressibility is cheap and L-independent. A *constant* depth $r^* = 4$ reproduces the edge string all the way to $L = 16$ — it does **not** grow with L. Physically the edge Majoranas are localised at the two ends, so building them costs ∼constant depth however long the chain between them is. This refines the earlier expectation ("depth must grow with L") from the χ-probe (§5): that statement is about the *full* wavefunction; the *edge-string diagnostic* escapes it.

Noise is the whole large-L story. Because r^* is fixed, the CNOT count grows only linearly, $n_{cnot} = r^*(L-1) = 4(L-1) = 6, 20, 28, 36, 44, 52, 60$, yet the noisy edge decays exponentially along $(1-p)^{n_{cnot}}$:

L	4	6	8	10	12	14	16
n_{cnot}	6	20	28	36	44	52	60
noisy $\langle X_0 Z_1 \cdots Z_{L-2} X_{L-1} \rangle$ (toy)	0.68	0.37	0.23	0.17	0.10	0.08	0.07

Under the toy $p = 0.05$ the diagnostic is effectively dead by $L \approx 14$.

16 Toy versus real hardware

The toy $p = 0.05$ is ∼6× harsher than a real device. The same 60 CNOTs under real Cairo ($2q$ error 7.5×10^{-3}) give $(1 - 0.0075)^{60} \approx 0.64$, i.e. $\langle X_0 Z_1 \cdots Z_{L-2} X_{L-1} \rangle \approx 0.6$ *at* $L = 16$ — the diagnostic *survives*. A device-calibrated run to $L = 10$ already shows this: noisy $\langle X_0 Z_1 \cdots Z_{L-2} X_{L-1} \rangle = 0.86, 0.76, 0.65, 0.61$ at $L = 4, 6, 8, 10$ versus the toy $0.68, 0.37, 0.23, 0.17$. So "noise kills the edge

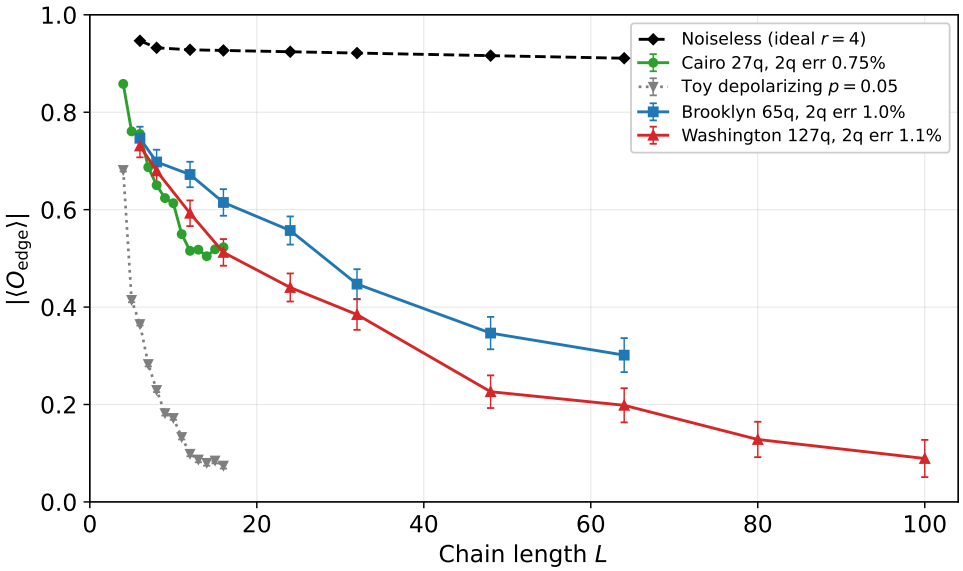

Figure 2: All fixed-depth ($r^* = 4$) edge-string results on one axis (`scripts/plot_largeL_overlay.py`). Colour coding matches the deck: **black** noiseless ideal (flat ∼0.9); **grey** toy depolarising $p = 0.05$ (dead by $L = 16$); **green** real Cairo 27q; **blue** Brooklyn 65q (to $L = 64$); **red** Washington 127q (to $L = 100$). Real devices survive far past the toy prediction and order by two-qubit error rate.

string by $L \approx 16$" is an artifact of the toy strength; on real IBM parameters it persists considerably further, and the honest large-L statement requires the `--backend` sweep.

16.1 The fixed-depth $L \to 100$ device sweep

To make the large-L statement honest we ran a dedicated fixed-depth ($r^* = 4$, no toy, no exact-GS reference) sweep on *two* real snapshots, `FakeBrooklynV2` (65q, so $L \leq 64$) and `FakeWashingtonV2` (127q, to $L = 100$). Because a single depth-2 tiling collapses the prepared state, the angles are **grown incrementally** in +2 steps with re-optimisation at each length, then the noisy edge string is sampled by the MPS trajectory estimator (`src/block4_largeL.py`, run via `scripts/largeL.slurm` on the Leipzig SC cluster). The noiseless edge string stays *flat* at ∼0.9 from $L = 6$ to $L = 100$: the finite-depth state is genuinely topological at every length, so the entire decay below is decoherence.

L	6	8	12	16	24	32	48	64	80/100
noiseless (ideal)	0.95	0.93	0.93	0.93	0.92	0.92	0.92	0.91	0.91/0.90
Brooklyn (1.0%)	0.75	0.70	0.67	0.61	0.56	0.45	0.35	0.30	—
Washington (1.1%)	0.73	0.68	0.59	0.51	0.44	0.38	0.23	0.20	0.13/0.09

The noisier chip (Washington, 2q error 1.15×10^{-2}) decays faster than Brooklyn (1.02×10^{-2}), the two **agree where they overlap**, and both extend the earlier Cairo run: Cairo (0.75%, gentler) sits at the top of the real-device band and cross-validates against the two new chips at $L \leq 16$. The edge string stays measurable to $L \sim 48$–64 and is essentially gone (∼0.09) by $L = 100$.

17 Conclusion

For the edge-string diagnostic the classical-simulation and NISQ questions come apart cleanly. The *reference* is free (free-fermion BdG, exact to hundreds of sites). The *ideal* VQE state is a mild 2^L object, computed without any 4^L matrix, and its expressibility for this observable is L-independent (constant $r^* = 4$). The genuine large-L wall is therefore neither ED nor expressibility but **decoherence over a linearly growing gate count** — and its severity is set by the real per-gate error rate, which for present IBM hardware is gentle enough that the topological diagnostic survives to noticeably larger L than the toy model suggests. The full wavefunction still obeys the harder wall of §5; the point is that the *measured* topological signature does not have to.